CUDA-Accelerated FP-Growth Frequent Itemset Mining

GPU weighted-support kernels in recursive FP-Growth: design, validation, and benchmarks

Student: Mashrafi Rodan
Course project: Frequent Itemset Mining
Institution: Stony Brook University
Prepared: October 28, 2026

Abstract

This project implements CUDA-accelerated FP-Growth in C++17 to enumerate all frequent itemsets above a user-defined minimum support. Transactions are compressed into a prefix-sharing FP-tree, and patterns are generated recursively from conditional pattern bases rather than by exhaustive candidate generation. The implementation outputs support counts, offers memory-conscious summary-only operation, and exports machine-readable CSV/JSON summaries. The CUDA variant computes weighted support histograms on the GPU for root and all recursive conditional FP-trees. C++ constructs and traverses tree structures.

Correctness is checked with a hand-worked five-basket dataset and an independent exhaustive enumerator on 100 randomized cases. Experiments use two locally available copies of UCI datasets (handwritten digits and Wisconsin Diagnostic Breast Cancer), a clearly labeled replicated scaling version of the digits sample, and separately generated synthetic retail transactions. The CPU execution times and memory usages in this report are measured Linux values, not estimates. CUDA host integration passed 34 simulator comparisons; real nvcc and GPU performance tests remain pending.

Implementation	Validation	Experimental scale
C++17 + recursive GPU histograms	7 CPU tests, 34 sim checks	569-53,910 transactions
CSV/JSON exact support output	100 CPU exhaustive oracle checks	Measured CPU 5-run medians

Validation status. The CPU baseline and simulated CUDA host interface have been exercised, but this environment has no NVIDIA GPU or nvcc. The device kernel and GPU timings must be tested on lab hardware. This is a CUDA/C++ **hybrid**, not a fully GPU-resident FP-tree.

1. Problem definition and support

Given a transaction database *D* with *N* rows, each transaction is a set of items. An itemset *X* is frequent when **support(X) = |{T in D : X is a subset of T}| >= min_count**. Fractional support *s* is converted to an absolute threshold **ceil(s N)**. The application accepts an absolute count such as `3`, a fraction such as `0.7`, or `70%`.

The output is the *complete* collection of qualifying non-empty itemsets, each with its exact transaction support. An item repeated twice within one row still counts only once. Dataset class labels are deliberately excluded, because this project is unsupervised pattern mining, not classification.

2. Implementation method

2.1 Input normalization and frequent 1-items

The program reads whitespace-separated item tokens, maps each distinct token to an integer ID, sorts and deduplicates every transaction, and counts singleton frequencies. Items below the minimum threshold are removed. Frequent items are then ordered by descending singleton support (with a deterministic ID-based tie rule).

2.2 Construction of the FP-tree

A root node represents the empty prefix. For each filtered, frequency-sorted transaction, the program walks existing child links where possible and increments their counters; when no child exists, it creates a new node. A header table stores each item’s overall support and pointers to all tree nodes bearing that item. Identical prefixes share nodes and counters.

2.3 Recursive conditional mining

Items in each tree are visited from lowest support to highest support. Each visited item is appended to the current suffix and emitted with its header support. The conditional pattern base is built by tracing the ancestors of every header-linked node; each ancestor path is weighted by that node’s count. A new conditional FP-tree removes items falling below support, and the procedure recurses until no frequent extension remains. This ordering prevents duplicate patterns.

Core pseudocode

```
mine(tree, suffix):
    for item in reverse(tree.frequent_items_by_support):
        pattern = suffix + [item]
        emit(pattern, tree.support[item])
        paths = []
        for node in tree.header[item]:
            paths.append((ancestors(node), node.count))
        conditional = build_fp_tree(paths, minimum_support)
        if conditional contains frequent items:
            mine(conditional, pattern)
```

The tree recursively decomposes the dataset into smaller projected problems. As in FP-Growth generally, the number of output patterns can still be exponential in the number of distinct items; no algorithm can avoid the cost of explicitly reporting exponentially many frequent subsets.

3. Small illustrative example: exact validation

The five-transaction market-basket example below was encoded directly as `data/toy_transactions.txt`. With minimum support count 3 (60% of the five transactions), the four singleton patterns and four pairs shown are the **only** frequent non-empty patterns.

ID	Items in transaction
T1	bread, milk
T2	bread, diaper, beer, eggs
T3	milk, diaper, beer, cola
T4	bread, milk, diaper, beer
T5	bread, milk, diaper, cola

3.1 The complete expected result

Frequent itemset	Support count
bread	4
milk	4
diaper	4
beer	3
bread, milk	3
bread, diaper	3
milk, diaper	3
beer, diaper	3

3.2 Worked conditional-pattern-base trace

For **beer** (support 3), the paths preceding its occurrences in the global FP-tree are `[bread, diaper] (1)`, `[milk, diaper] (1)`, and `[bread, milk, diaper] (1)`. Within these weighted prefix paths, **diaper** occurs 3 times, while bread and milk each occur only 2 times. At threshold 3, the conditional tree contains diaper only. FP-Growth therefore emits {beer, diaper} with support 3, and no other pair containing beer.

Independent Python combination-counting produces the same eight itemsets and the same exact support values. A three-item combination does not reach support 3 in these five baskets.

4. Testing and validation

The test suite runs the compiled C++ binary and independently computes every subset and its count in Python. The comparison is between complete itemset-to-support dictionaries, not just a few manually inspected outputs.

Automated check	What was verified
Five-basket example	All eight patterns and exact supports at count >= 3
Duplicate items in a row	Repeated occurrences do not inflate support
Disjoint / no valid pattern	Correct singleton-only or empty outputs
Absolute, fractional, %	Equivalent thresholds produce identical results
Dense / identical rows	All qualifying subsets emitted once
100 randomized data sets	C++ results exactly match independent enumeration
Repeated dataset	All support counts scale by factor 3; itemsets unchanged

Observed outcome: all seven test groups passed, including the 100 random generated instances (random seed 217). The random tests varied 2–8 distinct items, 3–18 transactions, and the minimum support; each compared complete dictionaries and rejected duplicate emissions.

4.1 Additional real-data consistency check

The 1,797-row UCI-derived handwritten-digits copy was replicated exactly 30 times to form 53,910 transactions. At 70% support, both datasets produced 3,457 distinct patterns; at 80%, both produced 481. The corresponding absolute minimum supports were 1,258 and 37,737 (70%) and 1,438 and 43,128 (80%), respectively. Replication is a scaling exercise and not a claim of 53,910 independent UCI observations.

4.2 Reproducibility

```
make
make test
python3 scripts/benchmark.py --repeats 5
./fp_growth data/toy_transactions.txt 3 \
    --output results/toy_patterns.csv \
    --summary results/toy_summary.json
```

Also completed: 34 full itemset/support comparisons between the CPU binary and the CUDA-enabled host control path using a CPU histogram simulator (four bundled datasets plus 30 randomized datasets). The simulator does not compile or execute the actual CUDA device kernel; real-device verification is still required. The Python scripts are only test/benchmark drivers, not mining implementations.

5. Benchmark results: executed CPU experiments

All rows below are **measured**, with support thresholds and outputs taken from the included `results/benchmark_results.csv`. Runtime is the median over five fresh processes, including input loading and mining. Peak RAM is the median of Linux process VmHWM readings, converted from KiB to MiB. Frequent-pattern counts and supports are exact. CPU times may vary on the lab computers.

Dataset	Rows	Support	Patterns	Time (s)	Peak RAM (MiB)
UCI-WDBC	569	40%	283	0.0121	5.00
UCI-WDBC	569	50%	30	0.0060	3.19
UCI-Digits	1,797	70%	3,457	0.0251	4.21
UCI-Digits	1,797	80%	481	0.0146	2.82
UCI-Digits x30	53,910	70%	3,457	0.2848	24.60
UCI-Digits x30	53,910	80%	481	0.2246	23.23
Synthetic retail (50k)	50,000	3%	265	0.3011	68.47
Synthetic retail (50k)	50,000	5%	108	0.2045	44.11
Synthetic retail (50k)	50,000	7%	60	0.1526	29.98

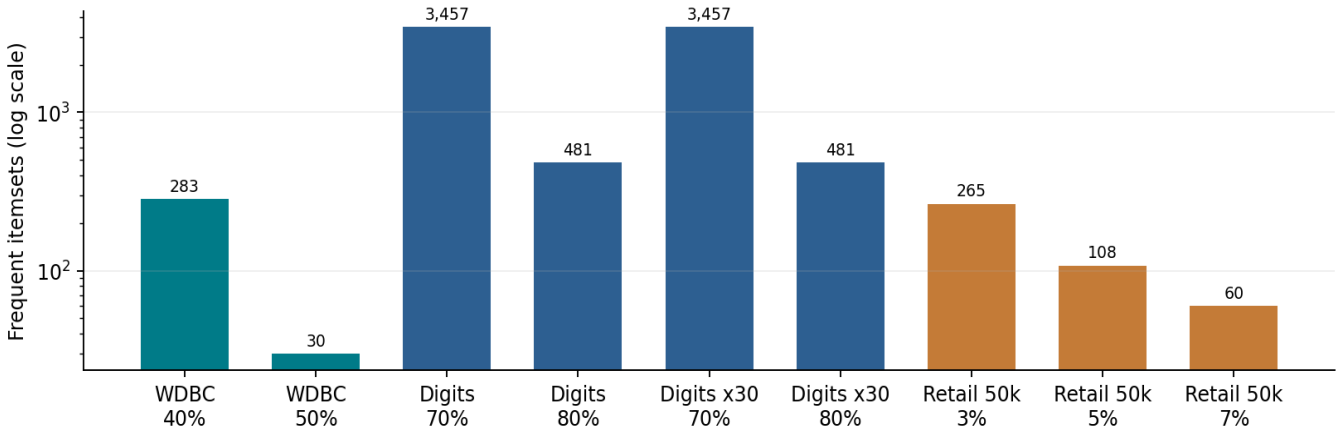


Figure 1. Pattern count versus support for each independent experiment (log-scaled y-axis; repeated digits rows are explicitly labeled).

6. Performance discussion

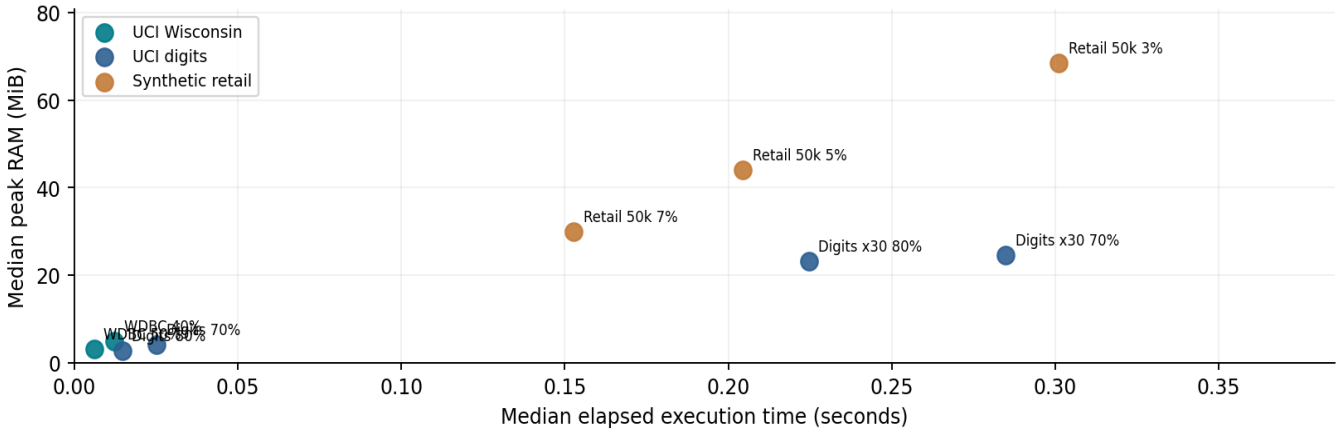


Figure 2. CPU time and peak resident memory for the nine measured scenarios. The same hardware was used for all nine experiments.

6.1 Effect of minimum support

The digits test-set copy yields 3,457 patterns at 70% but 481 at 80%; the number of output patterns therefore decreases as the support threshold rises. The breast-cancer copy drops from 283 patterns at 40% to 30 at 50%. On the synthetic retail stress test, increasing support from 3% to 7% reduces output patterns from 265 to 60 and drops median peak resident memory from approximately 68 MiB to 30 MiB in these observed runs.

6.2 Scaling and dataset density

The exact 30x repeated digits experiment retains the same frequent itemset identities at a fixed relative support, but the 53,910-row input increases parsing and tree-building work. The 50,000-row independent synthetic retail set has a different, sparsely selected item universe and uses more memory at low support because many conditional trees are constructed. Comparing runtimes between different datasets is not a controlled speed-up claim: row lengths, support levels, repeated prefixes, and item distributions all influence runtime.

6.3 Memory management and complexity

Conditional trees are built recursively and automatically released on return via C++ `unique_ptr`. The program does not retain every mined pattern in memory: full CSV results are streamed, and only a fixed-size top-K preview is retained. The reported RAM nevertheless includes input transactions, FP-tree nodes, header vectors, temporary weighted conditional paths, and runtime allocator overhead.

Initial singleton counting is linear in the number of item occurrences; per-row sorting adds sorting cost. Building and mining conditional FP-trees is data-dependent, and enumerating all frequent subsets requires exponential output size in the worst case. Peak RAM is not the sum of nodes allocated over the entire run; the summary reports both cumulative allocation and actual high-water memory.

7. CUDA design: GPU counting at every tree level

The CUDA build performs **weighted 64-bit support histogramming on the GPU at every recursive FP-tree level**. This is a substantial change from a root-only histogram prototype. A `SupportCounter` shared by the root tree and conditional-tree constructors flattens each group of weighted pattern-base paths into two arrays: item IDs and the path multiplicities. The device kernel uses one thread per flattened occurrence and `atomicAdd` to increment the item bucket by its weight.

7.1 Division of computation

CUDA device (GPU)	C++ host (CPU)
Root item supports	Parse input and remove duplicate row tokens
All conditional-tree item supports	Construct FP-tree nodes and headers
Parallel weighted integer atomic adds	Build weighted conditional pattern bases
64-bit histogram bucket accumulation	Recursively mine and emit unique patterns
Device-to-host histogram transfers	Write CSV/JSON and measure overall elapsed time

For an item i in a conditional pattern base P, **support(i) = sum of w(p), over all paths p in P containing i**. Using the original FP-tree node count as each path weight preserves exact conditional supports. The histogram result feeds standard FP-Growth filtering and tree construction. The host controls recursion because tree topologies are pointer-heavy and irregular; the GPU accelerates the regular, data-parallel support-count operation.

7.2 GPU dataflow (repeated at EVERY conditional level)

```
flatten(weighted_pattern_paths) -> ids[], weights[]
copy ids[] and weights[] from CPU to CUDA device
zero the device histogram
CUDA kernel: atomicAdd(histogram[ids[j]], weights[j])
copy support histogram back to host
CPU: filter frequent items, build conditional FP-tree
CPU: recursively mine extensions
```

GPU arrays are reallocated only if larger capacity is needed. The summary stores the GPU name, number of histogram kernel calls, number of weighted entries processed, and the maximum CUDA buffer allocation, which is **not** the total device memory usage including CUDA context.

8. Validation status and lab CUDA benchmarks

8.1 Exact CUDA verification: next lab session

```
make && make cuda
./fp_growth_cuda data/toy_transactions.txt 3 --no-output
python3 scripts/verify_cuda.py --random-cases 20
python3 scripts/benchmark_cuda.py --repeats 3
```

The CUDA verification script independently runs CPU and GPU builds, compares all emitted itemset-to-support dictionaries, checks GPU kernel-call counters, and independently exhaustively checks 20 generated small datasets. The benchmark compares CPU and CUDA on the same hardware with alternating process order, three timing replicates, and CPU/GPU peak memory reporting. It saves real measurements in `results/cuda_benchmark_results.csv`. **CUDA device measurements have not yet been run.**

Already tested (host-only)	Pending verification on NVIDIA GPU
All 7 CPU test groups passed	Compile the .cu file with nvcc
CPU: 100 random exhaustive cases passed	Compare every CUDA itemset and support
34 CPU vs GPU-path emulator runs matched	Run 20+ random GPU equality checks
9 datasets/thresholds benchmarked on CPU	Record GPU times and CUDA buffer memory

8.2 Tradeoffs, limitations, and future improvements

Each conditional histogram triggers host-to-device transfers and a GPU kernel launch. When conditional databases are small, transfer/launch overhead can dominate, so CUDA may be *slower* than the CPU despite parallel atomic additions. The hybrid miner still constructs and walks irregular tree structures on the CPU; it is not a completely GPU-resident implementation. A possible optimization is to batch multiple conditional supports into fewer launches.

The optional UCI Mushroom (8,124 rows), Chess (3,196), and Connect-4 (67,557) datasets can be downloaded with `scripts/get_uci.py`; these originals have not been benchmarked here. Existing measured inputs are UCI-derived Digits and Wisconsin cancer copies, replicated Digits for scaling, and a separately labeled synthetic retail dataset. Dense data at low support can generate exponentially many itemsets regardless of GPU throughput.

9. Conclusion

The project includes an exact C++ FP-Growth reference miner plus a CUDA hybrid that uses the GPU for weighted singleton histograms throughout root and recursive conditional FP-tree mining, not only at preprocessing. CPU and host-simulated CUDA paths passed tests; lab CUDA device compilation, correctness, and performance measurement remain necessary before claiming hardware-specific results.

10. Lab GPU measurement worksheet

Complete this worksheet from the real lab GPU benchmark output. The numbers below are deliberately blank because there was no CUDA device during project preparation. Copy actual data from `results/cuda_benchmark_results.csv` before final printing. A CPU/GPU speedup less than 1 means CUDA was slower.

Dataset	Min support	CPU (s)	CUDA (s)	CPU/CUDA
Toy baskets	3	_____	_____	_____
WDBC	50%	_____	_____	_____
Digits	80%	_____	_____	_____
Digits	70%	_____	_____	_____
Synthetic retail	7%	_____	_____	_____

GPU model: _____ **CUDA toolkit:** _____

CPU peak RAM: _____ MiB **CUDA peak RAM:** _____ MiB **CUDA buffer allocation:** _____ MiB

Verification: All real CPU-GPU itemset/support comparisons: PASS / FAIL (circle after testing).

References and dataset attribution

1. Han, J., Pei, J., & Yin, Y. (2000). Mining Frequent Patterns without Candidate Generation. *ACM SIGMOD*. https://doi.org/10.1145/342009.335372

2. UCI Machine Learning Repository, Optical Recognition of Handwritten Digits, dataset 80. https://archive.ics.uci.edu/dataset/80/optical+recognition+of+handwritten+digits

3. UCI Machine Learning Repository, Breast Cancer Wisconsin (Diagnostic), dataset 17. https://archive.ics.uci.edu/dataset/17/breast+cancer+wisconsin+diagnostic

4. scikit-learn documentation: load_digits and load_breast_cancer (bundled UCI-derived data copies). https://scikit-learn.org/stable/modules/datasets/toy_dataset.html

5. UCI Machine Learning Repository: Mushroom (#73), Chess King-Rook vs King-Pawn (#22), Connect-4 (#26). https://archive.ics.uci.edu/

Measured CPU hardware: INTEL(R) XEON(R) PLATINUM 8573C; Linux-6.18.55-x86_64-with-glibc2.41; g++ (Debian 14.2.0-19) 14.2.0. Times and RAM values reflect this execution environment and are not fixed algorithm constants.